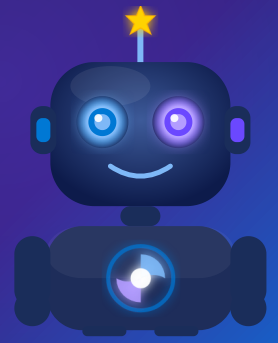


GitHub Copilot ハンズオン

GitHub Copilot の基本操作から演習まで、
Visual Studio Code を使って体験してみましょう。

VS Code × GitHub Copilot



ここでは、GitHub Copilot の基本的な操作方法を中心に、簡単な演習を通じて体験していただきます。GitHub Copilot の実行環境として、**Visual Studio Code (VS Code)** を使用します。VS Code には GitHub Copilot が統合されているため、追加の拡張機能をインストールせずに利用できます（2026年4月に完全統合）。VS Code は汎用的なエディターで、コーディングだけでなく文章作成など、さまざまな用途に活用できます。



STEP 0 — 事前準備（10分）



まずは、VS Code をインストールして、GitHub Copilot をつかえる状態にしましょう。

1 Visual Studio Code (VS Code) をインストール

<https://code.visualstudio.com>

2 VS Code を開く

- 方法1: スタートメニューから VS Code を起動して、**File** → **Open Folder** でこのフォルダを選択
- 方法2: Terminal でこのフォルダに移動して、**code .** と入力して Enter
- 方法3: Explorer でこのフォルダを開き、アドレス欄に **code .** と入力して Enter ★オススメ

3 GitHub Copilot の有効化

- VS Code 右下の GitHub Copilot アイコンをクリック
- 初回だけ **Enable AI Features** → **Trust Folder & Continue** を選択
- ブラウザで認証画面が表示されたら自分の GitHub アカウントでサインイン&承諾



STEP 1 — VS Code 上での GitHub Copilot

VS Code 組み込みの GitHub Copilot 機能は、主に下記の 2 種類の操作を起点として動作します。

起動方法		特徴
● インライン補完機能	自動（ファイル編集）	ファイルを編集しているだけで、リアルタイムに自動補完されます。補完内容は <code>Tab</code> で採用、 <code>Esc</code> で却下します。
● チャット機能	<code>Ctrl+Alt+I</code> （または <code>Ctrl+I</code> でインラインチャット）	チャットエリアを表示し、会話形式で任意の指示を出します。生成内容を確認しながら繰り返し修正できます。



STEP 2 — 基本的な使い方

インライン補完機能に関しては勝手に補完されるのを待つだけでよいので、ここではチャット機能を中心に説明します。

基本操作

最低限、下記の操作を覚えれば、チャット機能を使った編集作業ができるようになります。

操作	キー
チャットパネルを開く / 閉じる	<code>Ctrl + Alt + I</code>
エディタ上でのインラインチャット（無効な場合あり）	<code>Ctrl + I</code>
提案コードを適用	<code>保持</code> ボタンをクリック
過去履歴をクリア	チャットで <code>/clear</code> と入力
新しい会話を開始	チャット上部の <code>+</code> ボタンをクリック

使い方のポイント

💡 生成内容を確認して修正を繰り返す

チャットで指示を出すと、自動生成されたコードや文章の適用待ち状態になります。Copilot の提案は無条件に採用せず、**内容を確認してから** **保持** **ボタン** をクリックします。気に入らなければ追加指示を出して何度でもやり直せます。

💡 ファイルの参照を追加する

チャット入力欄の **+** **アイコン** を押すか、ファイルをドラッグ&ドロップすると、そのファイルの内容をコンテキストに含めることができます。一時的に追加情報を与えて作業指示を出したいときに便利です。

💡 ファイルの一部を処理

エディタ上で範囲選択した状態でチャット入力すると、その内容がコンテキストに乗ります。チャットパネルが開いていないときは、**範囲選択** → **Ctrl + Alt + I** と入力することで、選択範囲の内容をコンテキストに含めた状態ですぐに指示を出せます。特定範囲の **文章の校正**、**文章の要約**、**コード解説**、**リファクタリング** など、さまざまな場面で活用できます。

💡 別の処理をしたいときは会話履歴をクリアするか新しいセッションを開始する

チャットで連続して指示を出すと、過去の会話履歴（コンテキスト）を考慮してエージェントが動作するため、LLM への入力トークンがどんどん増えていきます。これまでの会話履歴をクリアして、新しい作業を開始したいときは、チャット欄に **/clear** と入力します。これにより、これまでと関係のない会話履歴が混ざるのを防げます。LLM への入力トークン数も減るため、利用コストの削減にもつながります。

これまでの会話履歴を残したまま新しいセッションを開始したいときは、チャット上部にある **+** **ボタン（新しいチャット）** を押します。過去のセッションはセッションパネルで一覧表示でき、いつでも再開できます。



STEP 3 — 演習



実際に手を動かして GitHub Copilot のいろいろな使い方を試してみましょう。

演習 1 インライン補完で関数を生成する

Python のソースコードファイル (`exercise/main.py`) を開いて、インライン補完機能で「フィボナッチ関数」を生成してください。

操作例

1. 左のエクスプローラーから `main.py` を新規作成
2. 任意の行に `# フィボナッチ関数` というコメントを入力
3. しばらくすると関数の内容が薄く表示されるので `Tab` で採用

💡 ヒント

補完機能が動作しない場合は、`Ctrl + I` や `Ctrl + Alt + I` でチャットパネルを開いて「実装して」と指示を出してしまうのが早いです。

演習 2 選択範囲の文章を修正する

下記のようなまとまりのない議事録が Markdown ファイル (`exercise/議事録.md`) に書かれています。アクションアイテムを抽出し、担当者と期限を明確にした表に変換してください。



議事録

開催日

2026年8月5日 19:00-19:30

メモ

- 連休前なので臨時の報告会として開催。
- カメラで撮影した写真をテレビに表示するサービスの開発状況を確認したと田中さんが報告。転送遅延の問題が引き続き課題。
- 遅延原因の調査結果を今週中にまとめて共有してほしい。
- UI デザインのたたき台は鈴木さんが担当。リモコン操作の画面遷移が未確定なので、来週中には案を出す。
- 対応テレビの機種リストが古いため、山田さんが最新モデルへの対応可否を確認して更新する（今月中）。
- 次回ミーティングは再来週に設定。それまでに各自タスクを進めておく。

操作例

1. VS Code 上で `.md` ファイル内の該当箇所のテキストを範囲選択
2. `Ctrl + I` を押してインラインチャットを開く（場合によっては独立したチャットウィンドウがアクティブになります）
3. 次のように指示：



アクションアイテムを抽出し、内容・担当者・期日(YYYY-MM-DD)を表にまとめてください。

演習 3 スラッシュコマンドを使う

チャットエリアにスラッシュコマンド (`/explain`) を入力すると、難しいコードの解説をしてもらえます。同様の指示を文章入力で行うこともできますが、こういったよく使う指示は組み込みのスラッシュコマンドとして用意されています。

操作例

1. Python コードの任意の範囲を選択（演習 1 で作ったフィボナッチ関数など）
2. `Ctrl + Alt + I` → `/explain` と入力
3. チャットエリアに詳しい解説が表示されます

演習 4 カスタムスラッシュコマンド（カスタムプロンプト）を作る

よく使うプロンプトは、カスタムプロンプト (`.github/prompts/*.prompt.md`) として定義しておくことができます。スラッシュ (`/`) 始まりのコマンドとして呼び出すことができるので、「スラッシュコマンド」とも呼ばれます。

カスタムプロンプト用のファイルは手動で作成してもよいし、`/create-prompt` スラッシュコマンドで自動生成することもできます（自動生成してから内容を修正するのがおすすめです）。ここでは、日本語文章を校正する `/revise` コマンドを作ってみましょう。

操作例

まず、`/create-prompt` スラッシュコマンドを使って、カスタムプロンプトの雛形ファイルを作成します。チャット欄に、次のように入力します。



/create-prompt 日本語文章の校正を行う revise スラッシュコマンドを日本語で作成してください。「。の後は必ず改行」というルールを含めてください。

例: [.github/prompts/revise.prompt.md](#) (自動生成結果をマニュアル修正したもの)



```
---  
name: revise  
description: 日本語文章を自然で読みやすい表現に校正します。  
argument-hint: 校正したい日本語文章を入力  
agent: agent  
---
```

入力された日本語文章を校正してください。

要件:

- 意味は変えず、誤字・脱字・不自然な言い回しを修正する。
- 分かりにくい文章は、適宜説明文を補う。
- フォーマルになりすぎない、自然で読みやすい日本語にする。
- 「。の後は必ず改行」する。

実行手順:

- エディタで選択されている範囲がある場合は、その選択範囲を校正後本文で直接置換する。
- 選択範囲がない場合は、ユーザーが入力した本文を校正対象として扱う。

使い方

1. エディタ上で、校正したい範囲を選択
2. `Ctrl + I` あるいは `Ctrl + Alt + I` でチャットパネルを開き、`/revise` と入力して Enter
3. エディタ上に指示どおり校正された文章が表示されます。内容がよければ、「保持」ボタンを押して反映します。

演習 5 カスタムインストラクションを設定する

プロジェクト内で守るべき記述ルールや、常に守ってほしい方針がある場合は、下記のファイルにカスタムインストラクションとして記載します。

例: [.github/copilot-instructions.md](#)



- お嬢様口調で話してください。
- Python を使用する場合は、`pip` や `python` コマンドを使わずに、`uv add` や `uv run` などの `uv` コマンドを使ってください。

💡 必要最小限にすること

この情報は、すべての会話のコンテキストに組み込まれるため、必要最小限にすることが推奨されています。 不要な情報を含めると、入力トークンが増えてコストが上がり、LLM の応答精度が低下する原因になります。

特定のファイルにのみ適用したいルールは、下記のような専用ファイル
(`.github/instructions/*.instructions.md`) で定義します。

例: `.github/instructions/python.instructions.md`



```
---  
applyTo: "**/*.py"  
---
```

Python Coding Instructions

Style

- Use Python 3.10+ type hints with built-in generics: list, dict, tuple, set.
- Prefer clear, intention-revealing names for functions and variables.
- Keep each function focused on a single responsibility.

Documentation

- Add docstrings to all public functions and classes.
- For non-trivial logic, add short comments explaining why, not what.

演習 6 Plan モードで計画を立てる

GitHub Copilot はデフォルトで Agent モードに設定されているため、指示を出すとすぐにファイルの編集作業を始めようとします。 ある程度複雑な作業を行う場合は、まず「**Plan モード**」に切り替えて作業計画を立て、そのうえでエージェントに自律的な作業を任せると、後戻りが減って効率的です。 手順が明確になっているため、ワンショットで作業を終えられる可能性も高まります（ご飯を食べに行って帰ってくると想定通りのものが完成している）。

操作例

1. チャットパネル下部の「エージェントの設定」メニュー（Agent/Ask/Plan のいずれかが表示）から、**Plan** モードに切り替えます。
2. チャット欄に、計画を立てたい作業内容を入力して Enter を押します。
3. 作業計画の提案が表示されたら、必要に応じて情報を補い、計画をブラッシュアップします。
4. 計画ができたなら「Autopilot を使用して開始する」をクリックすると、計画に沿った作業を自動で進められます。

 /plan スラッシュコマンドでも切り替え可能

現在のモードに関係なく計画用の指示を出せるので、慣れるとこちらの方が便利です。



```
/plan 1つのHTMLファイルで構成された、シンプルなブロック崩しゲームを作成して。
```

演習 7 カスタムエージェント（ペルソナ）を作る

TBD

この演習は現在準備中です。

演習 8 便利な Skills を作る

TBD

この演習は現在準備中です。

おまけ — ターミナルから使える Copilot CLI

VS Code を開かずにターミナルだけで Copilot を使いたい場合は、GitHub Copilot CLI を利用します。コードの編集が主目的ではなく、汎用的なファイル操作を行いたいときに便利です（フォルダ内のファイル整理など）。

1. GitHub Copilot CLI をインストール（Windows の場合）



```
winget install GitHub.Copilot
```


2. ターミナルから Copilot CLI を起動



```
cd <プロジェクトのルートディレクトリ>  
copilot
```

最初に GitHub アカウントでサインインすると、チャット形式で指示を出せるようになります。

使用例

- ディレクトリ内の画像ファイルを `img-0001.png`、`img-0002.png` のような連番にリネームして
- GitHub 上にリファクタリング用の Issue を作成して
- `Revert the last commit, leaving the changes unstaged`
- `List my open PRs`

Claude Code をご利用の方へ

GitHub Copilot CLI の操作方法は Claude Code とほぼ同じです。そのため、これまでのノウハウをそのまま活用できます。

- `Shift + Tab` でモード切り替え
- `Esc` で割り込み指示
- `/` でスラッシュコマンドを入力